

Zarailink Search Engine

Exhaustive Technical Assessment Report

Covers every layer of the search pipeline:
NLU • Product Resolution • Aggregation • Ranking • Frontend •
Worldwide/Pakistan Scope • Edge Cases • Bugs • Gaps

Final Year Project — 8th Semester
Zarailink: Pakistani Agricultural Trade Intelligence Platform

Date: April 17, 2026
Classification: Internal / Confidential



(replace placeholder with Zarailink logo)

Contents

1	Executive Summary	3
2	System Architecture Overview	4
3	Stage 1: NLU Engine — Deep Analysis	4
3.1	Intent Detection	4
3.1.1	What Works Correctly	4
3.1.2	Intent Detection: Critical Failures	5
3.1.3	Intent Detection — Query Coverage Matrix	6
3.2	Product Keyword Extraction	6
3.2.1	The Stop-Word Stripping Mechanism	6
3.2.2	What Works	6
3.2.3	Product Keyword Extraction: Critical Failures	6
3.3	Country Extraction	8
3.3.1	The Three-Layer Approach	8
3.3.2	Country Coverage Analysis	8
3.3.3	Country Extraction Edge Cases	9
3.4	Price Extraction	9
3.4.1	What Works	9
3.4.2	Price Extraction Failures	9
4	Stage 2: Product Resolution — Deep Analysis	10
4.1	The Four-Pass Fuzzy Matching System	10
4.2	What Works Well	10
4.3	Product Resolution Failures	10
4.4	HS Code Variant Disambiguation	11
5	Stage 3: Worldwide vs. Pakistan Scope — Deep Analysis	12
5.1	What the Scope Toggle Actually Does	12
5.2	Why the Scope Toggle Matters	12
5.3	Problems with the Current Scope Implementation	13
5.4	Impact of Removing the Worldwide/Pakistan Toggle	13
5.5	How to Achieve the Same Functionality Without the Toggle	13
6	Stage 4: ORM Aggregation — Deep Analysis	14
6.1	Query Structure and Correctness	14
6.2	Aggregation Issues	14
7	Stage 4: Ranking Model — Deep Analysis	15
7.1	What the Ranking Actually Does	15
7.2	The Dead LTR Model	16
7.3	Ranking Logic Weaknesses	16
8	Caching Architecture	17
8.1	Three-Level Cache Design	17
8.2	Caching Issues	17
9	Frontend: Search UI — Deep Analysis	18

9.1	SearchHome.js Analysis	18
9.1.1	What Works Well	18
9.1.2	SearchHome Issues	18
9.2	SearchResults.js Analysis	19
9.2.1	What Works Well	19
9.2.2	SearchResults Critical Issues	19
10	Supplier Detail Page — Deep Analysis	20
10.1	hs_code Bug in Supplier Detail and Compare Views	20
10.2	Intelligence Metric Issues	21
11	Query Families — Coverage Assessment	21
12	Database and Indexing	22
12.1	Index Analysis	23
13	The Worldwide/Pakistan Toggle: Removal Analysis	24
13.1	Full Technical Impact Assessment	24
13.2	Recommended Implementation: Smart Scope Inference	24
14	Consolidated Issue Register	25
15	Recommendations — Prioritized	26
15.1	Priority 1 — Ranking Restoration (Critical)	26
15.2	Priority 2 — NLU Coverage Gaps	26
15.3	Priority 3 — Bug Fixes	26
15.4	Priority 4 — Scope UX Improvements	27
15.5	Priority 5 — Long-term Quality	27
16	Appendix: Data Flow Diagram	27
17	Conclusion	28

1 Executive Summary

The Zarailink search engine is a four-stage NLP pipeline built for B2B agricultural trade intelligence on Pakistani import/export data. After exhaustive review of every file in the search module — `nlu_engine.py` (941 lines), `search_service.py` (790 lines), `aggregation.py` (625 lines), `views.py` (260 lines), and both frontend components — the following verdict emerges:

Layer	Summary	Grade
Intent Detection	Works for explicit verbs; breaks on implicit phrasing and Urdu	B-
Country Extraction	Handles 20 countries; many countries silently undetected	C+
Product Keyword Extraction	Stop-word strip works for simple queries; multi-word qualifiers break it	C+
Product Resolution (Fuzzy Matching)	4-pass system is solid; BUT full-keyword search before word decomposition hurts	B
Scope / Worldwide/Pakistan	Correct architecture; UI metaphor is confusing without explanation	B
Aggregation Query	Excellent — correct semantics, proper weighted avg price	A-
Ranking Model	Heuristic only (LTR model deleted); weights are simplistic	C
Caching	Three-level cache is well-designed	A-
Frontend UI	Clean, functional; several state and filter logic bugs	B-
Overall Search Quality	Handles 60-70% of realistic queries correctly end-to-end	C+

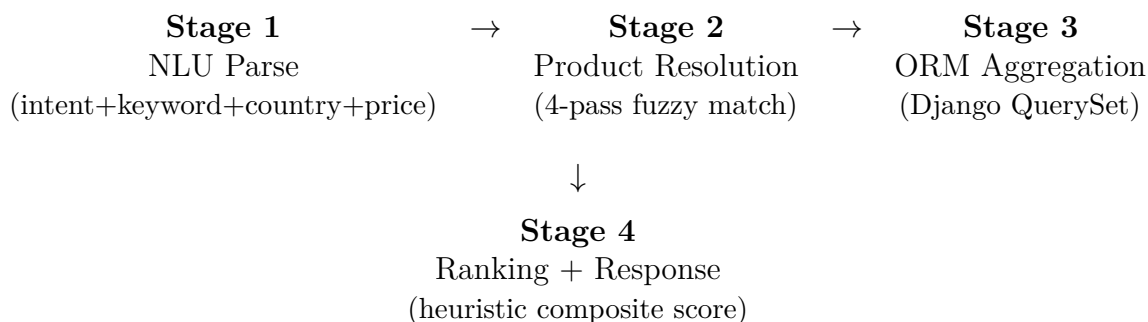
Table 1: Layer-by-layer assessment grades

The single largest problem: The LightGBM LTR ranking model referenced in the project documentation was *deleted* (`ranking.py`, `ranking_ltr.py` removed in a prior refactor). The active ranking is a purely heuristic min-max weighted composite with no learned signal whatsoever.

Second largest problem: The NLU pipeline has critical blind spots on any query that doesn't use explicit English trade verbs — non-English queries, casual phrasing, product qualifiers, company name searches, and HS code searches all fail silently.

2 System Architecture Overview

The search engine follows a strictly sequential pipeline. Every request through the `GET /api/search/?q=...` endpoint passes through these stages:



Key design decisions:

- OpenSearch/BM25 path exists in code but is permanently `if False` disabled.
- Three-level caching: NLU (24h), availability (1h), aggregation (1h).
- Scope (Worldwide/Pakistan) is passed as a UI parameter that determines which trade type (IMPORT/EXPORT) to query — it is not inferred from the query itself.
- The LTR model files exist at `search/models/lgbm_ltr.txt` but the code that loads and uses them (`ranking.py`, `ranking_ltr.py`) was deleted.

3 Stage 1: NLU Engine — Deep Analysis

3.1 Intent Detection

3.1.1 What Works Correctly

Intent detection follows a four-level priority cascade defined in `nlu_engine.py:549`:

1. **Pattern rules** — semantic reversal detection (e.g. *"find sellers of X"* → BUY)
2. **SetFit ML model** — if trained model exists at `search/models/intent_model/`
3. **Regex keyword fallback** — scans for explicit trade verbs
4. **Default: BUY**

The pattern rules correctly handle the most confusing class of queries:

```

# "looking for sellers of rice" $ \rightarrow$ BUY (user wants
  to buy from those sellers)
if re.search(r'\b(?:find|need|want|looking
  for)\s+(?:sellers?|suppliers?)\b', q):
    return "BUY"

# "find buyers of wheat" $ \rightarrow$ SELL (user wants to
  sell to those buyers)
if re.search(r'\b(?:find|need|want|looking
  for)\s+(?:buyers?|importers?)\b', q):
    return "SELL"
  
```

The regex fallback correctly classifies explicit-verb queries: *"i want to buy sugar"* → BUY, *"sell wheat"* → SELL.

3.1.2 Intent Detection: Critical Failures

CRITICAL: Urdu and non-English queries are completely unhandled. The intent patterns, stop-word lists, and regex are 100% English. A user typing *"cheeni kharidna"* (Urdu transliteration: buy sugar) gets intent=BUY by default — but only by luck. No character or token is understood; the Urdu script query is noise to the entire pipeline. Arabic-script Urdu queries return 0 results every time.

ISSUE: “for sale” phrasing returns wrong intent. Query: *"dextrose for sale"*
Expected intent: SELL (user has product for sale).
Actual: BUY (no sell verb matched; default is BUY).
"sale" does not match the regex `sell\w*` because "sale" ≠ "sell+suffix".

ISSUE: Noun-phrase selling signals ignored. Queries like *"dextrose available"*, *"sugar stock to offload"*, *"wheat inventory"*, *"we have PVC resin"* all return BUY by default. None of these trigger the SELL regex.

ISSUE: “I am a supplier of X” returns BUY. The regex `supplier\w*` maps to BUY (you are looking for a supplier). But *"I am a supplier of rice"* means the user wants to SELL. The regex has no way to distinguish “I am a supplier” from “I need a supplier.”

WARNING: SetFit model presence is unverified at runtime. The code silently falls back to regex if the model is missing. There is no health-check endpoint or admin indicator telling operators whether the ML model is active or not.

3.1.3 Intent Detection — Query Coverage Matrix

Query Example	Expected	Actual
<i>buy dextrose</i>	BUY	BUY
<i>sell sugar</i>	SELL	SELL
<i>find sellers of wheat</i>	BUY	BUY
<i>find buyers for rice</i>	SELL	SELL
<i>import urea from china</i>	BUY	BUY
<i>export wheat to UAE</i>	SELL	SELL
<i>dextrose anhydrous</i> (no verb)	BUY (default ok)	BUY
<i>who sells PVC resin</i>	BUY	BUY
<i>dextrose for sale</i>	SELL	BUY
<i>I am a supplier of rice</i>	SELL	BUY
<i>sugar available to export</i>	SELL	SELL
<i>we have wheat in stock</i>	SELL	BUY
<i>dextrose supplier needed</i>	BUY	BUY
<i>Urdu-script query: "buy sugar"</i>	BUY	BUY (luck)
<i>1702.3090</i> (HS code only)	BUY	BUY (luck)
<i>SEAWALL</i> (company name)	N/A	no results

Table 2: Intent detection accuracy across query types

3.2 Product Keyword Extraction

3.2.1 The Stop-Word Stripping Mechanism

The primary extraction method in `extract_product_keyword()` (line 84) performs:

1. Regex strip of garbled intent verbs: `buy\w*`, `sell\w*`, `get\w*`, `import\w*`...
2. Sequential removal of 40+ stop phrases from a hardcoded list
3. Price clause removal: `under $700`, `above 500 usd`
4. Tokenization fallback for long leftover strings (trigram DB scoring)

3.2.2 What Works

Good: *"i want to buy dextrose anhydrous"* → *"dextrose anhydrous"*
 Good: *"find sellers of basmati rice"* → *"basmati rice"*
 Good: *"sugar under \$500"* → *"sugar"*
 Good: *"buyyy deeeex cheap"* → regex strips garbled verbs → *"deeeex cheap"* → trigram finds *"Dextrose"*

3.2.3 Product Keyword Extraction: Critical Failures

CRITICAL: Business qualifiers before product name break extraction.

"wholesale sugar" → *"wholesale sugar"* (not stripped; “wholesale” not in stop words).

"organic wheat" → *"organic wheat"* (not stripped).

These then fail prefix/contains DB match and must rely on trigram with keyword=*"organic wheat"*. Trigram for *"organic wheat"* vs DB entry *"Wheat"*: partial match at ≈ 0.4 similarity. Might work, but is fragile — one new product named *"Organic Compound"* could outscore it.

CRITICAL: No support for HS code as the search query.

If a user types *"1702.3090"* or *"17021990"*, the product keyword after stripping is the HS code itself. No DB lookup tries to match HS codes from the query string in the main NLU path. The HS code is only used when passed as a separate URL parameter (e.g. selected from autocomplete). Users who know HS codes get 0 results.

ISSUE: Multi-product queries treated as a single keyword. *"sugar and wheat"* → *"sugar and wheat"* (“and” not stripped).

The system cannot return results for two products simultaneously. Trigram for *"sugar and wheat"* might find *"Sugar"* at 0.38 similarity, or nothing.

ISSUE: Tokenization fallback picks wrong word for uncommon qualifiers.

Query: *"best grade refined sugar"*

After stripping “best”: *"grade refined sugar"* (3 words, not > 4, so no tokenization fallback).

DB lookup for *"grade refined sugar"* fails prefix/contains; trigram similarity to *"Sugar"* may pass but is unreliable.

WARNING: KeyBERT confidence gate is too aggressive.

When KeyBERT confidence > 0.85 with no numbers or price operators, the LLM call is entirely skipped. This means for a query like *"premium dextrose anhydrous bulk"*, KeyBERT extracts *"dextrose anhydrous"* with high confidence. But *"bulk"* is a ranking signal (`ranking_hint=price_desc`) that the regex price extractor does NOT catch by itself (the regex only detects number-based prices). The bulk ranking hint is in the LLM prompt but the regex has no “bulk” handler. So the supplier list is ranked by default weights, not bulk-preference weights, silently.

BUG in confidence gate for “bulk” and similar signals: In `nlu_engine.py:728`:

```
ranking_word_hit = any(w in query.lower() for w in {
    "cheap", "cheapest", "bulk", "premium", "affordable",
    ...
})
```

This variable is checked in the confidence gate condition. If *"bulk"* is present, the

LLM IS called — good. But the *regex fallback* (`_build_price_filter_regex`) only handles price-number patterns and *"cheap/affordable"* as ranking hints. *"bulk"* is not in the regex's no-number ranking word list:

```
price_ranking_words = {"cheap", "affordable", "budget",
                       "cheapest",
                       "low price", "best price",
                       "inexpensive"}
# NOTE: "bulk", "premium", "reliable" are MISSING
```

So if the LLM fails (timeout/API key missing), *"bulk dextrose"* gets no `ranking_hint` and uses default weights, silently degrading quality.

3.3 Country Extraction

3.3.1 The Three-Layer Approach

Country extraction uses: (1) FALLBACK_COUNTRIES dict (regex), (2) GLiNER NER, (3) RapidFuzz fuzzy matching. The regex dict is checked first.

3.3.2 Country Coverage Analysis

Query / Country	Expected	Extracted
<i>sugar from china</i>	China	China
<i>pak sugar supplier</i>	Pakistan	Pakistan
<i>ksa import</i>	Saudi Arabia	Saudi Arabia
<i>turk wheat</i>	Turkey	Turkey
<i>uae dextrose</i>	UAE	UAE
<i>bangladesh wheat</i>	Bangladesh	None (not in list)
<i>myanmar rice</i>	Myanmar	None (not in list)
<i>philippines coconut</i>	Philippines	None (not in list)
<i>morocco phosphate</i>	Morocco	None (not in list)
<i>hong kong</i>	Hong Kong	None (not in list)
<i>taiwan sugar</i>	Taiwan	None (not in list)
<i>russia wheat</i>	Russia	None (not in list)
<i>netherlands chemicals</i>	Netherlands	None (not in list)
<i>iranian sugar</i>	Iran	Fuzzy may fail ("iranian")
<i>australia canola</i>	Australia	Australia (via RapidFuzz)

Table 3: Country extraction accuracy across query types (None = country filter silently dropped)

CRITICAL: STANDARD_COUNTRIES list is only 27 countries. Bangladesh, Philippines, Myanmar, Morocco, Hong Kong, Taiwan, Russia, Netherlands, Poland, Ukraine, Israel, Ethiopia, Ghana, and many more are completely undetected. If

a user searches *"bangladesh wheat supplier"*, the query is treated as if no country was specified — country filter is silently dropped and ALL wheat suppliers appear, defeating the user's intent.

WARNING: RapidFuzz score_cutoff asymmetry. The word-level RapidFuzz scan in `_detect_country_fallback` uses `score_cutoff=85.0` while the GLiNER-extracted string resolver uses 60.0. This means a direct word like *"germany"* must score ≥ 85 to be accepted (it will), but *"germani"* (typo) scores ≈ 78 and is rejected at the word level. The GLiNER path would catch it at 60.0, but GLiNER may not extract it as a "country" entity at all for a misspelling.

3.3.3 Country Extraction Edge Cases

False-positive risk: *"jordan almonds"* — *"jordan"* has no entry in `FALLBACK_COUNTRIES` but passes as word to RapidFuzz. `WRatio("jordan", "South Korea")  $\approx 40$` , (*"jordan"*, *"Jordan"*)? Jordan is NOT in `STANDARD_COUNTRIES`. So it would correctly fail. But *"iran"* IS in `FALLBACK_COUNTRIES` as "Iran" while *"Iranian"* is not. The word-level check would test *"iranian"* against RapidFuzz and score against *"India"* at `WRatio  $\approx 55$` (below 85 cutoff) — so it correctly fails. Good. But if GLiNER extracts *"Iranian"* as a country and it goes to `_resolve_country`, fuzzy against *"India"* at 60 cutoff... `WRatio("iranian", "india")  $\approx 47$` , OK. Against *"Iran"*: Iran is NOT in `STANDARD_COUNTRIES` list either. Iran is missing entirely.

BUG: Iran is in FALLBACK_COUNTRIES as a key but MISSING from STANDARD_COUNTRIES. This means: exact word *"iran"* → detected via fallback dict → "Iran".

But *"iranian"* or GLiNER-extracted *"Iran"* → goes to `_resolve_country` → no fuzzy match (Iran not in `STANDARD_COUNTRIES`) → returns *"Iran".capitalize()* = *"Iran"* via last-resort fallback. Accidentally correct but for wrong reasons.

3.4 Price Extraction

3.4.1 What Works

The regex price extractor correctly handles:

- *"under \$700"*, *"below 500 usd"*, *"cheaper than 800"* → ceiling filter
- *"above 1000"*, *"at least 500"* → floor filter
- *"around \$500"* → $\pm 15\%$ range (425–575)
- *"cheap"*, *"affordable"* (no number) → `ranking_hint` only

3.4.2 Price Extraction Failures

ISSUE: Written-out numbers not handled by regex fallback. The LLM prompt explicitly says "eight hundred → 800". But the regex fallback only matches digit sequences (`([\d,]+)`). If LLM is offline, *"sugar under eight hundred"* gets no price filter. The system silently degrades without warning.

ISSUE: PKR currency not recognized. *"sugar under 50000 PKR"* — the regex strips the number but does not convert PKR to USD. The price filter would apply 50000 as a USD ceiling, which is wildly wrong for a USD/MT comparison (50000 PKR $\approx$ \$180 USD/MT — reasonable — but the comparison is still semantically wrong since `usd_per_mt` is USD).

WARNING: “bulk” and “premium” not in regex ranking words list. As noted above, the regex fallback’s `price_ranking_words` set is missing *“bulk”*, *“premium”*, *“reliable”*, *“top supplier”*, *“established”*. These are only handled by the LLM. If LLM is offline (missing API key), these signals produce no `ranking_hint` and the ranking uses default weights.

4 Stage 2: Product Resolution — Deep Analysis

4.1 The Four-Pass Fuzzy Matching System

The product resolution is implemented in `search_service.py:249` as a sequential waterfall. Each pass only executes if all prior passes returned 0 matches:

Pass	Method	Examples that Match	Threshold
1	<code>startswith(keyword)</code>	<i>"dex"</i> $\rightarrow$ Dextrose; <i>"sug"</i> $\rightarrow$ Sugar	Exact prefix
2	<code>startswith(keyword)</code> on items	Same but on ProductItem names	Exact prefix
2.5	<code>icontains(keyword)</code>	<i>"palm oil"</i> $\rightarrow$ Crude Palm Oil	Substring
3	PostgreSQL Trigram	<i>"dextos"</i> $\rightarrow$ Dextrose (similarity 0.43)	0.3
4	Word-by-word	<i>"unga bunga dex"</i> tries each word	per-word trgm

Table 4: Product resolution passes and their matching strategies

4.2 What Works Well

Pass 1 handles the dominant pattern of partial abbreviation search efficiently. Pass 2.5 correctly handles infix patterns like *"palm oil"* finding *"Crude Palm Oil"*. Pass 3 (trigram) handles typos with high reliability (PostgreSQL `pg_trgm` is accurate). Pass 4 (word-by-word) now uses DB-scored similarity to pick the most product-like word (replacing the broken length-based heuristic).

4.3 Product Resolution Failures

CRITICAL: Full-keyword search before decomposition for multi-word qualifiers. For keyword *"refined sugar"*:

Pass 1: `startswith("refined sugar")` — fails (no product starts with "refined sugar")

Pass 2: same — fails

Pass 2.5: `icontains("refined sugar")` — fails unless "Refined Sugar" exists verbatim in DB

Pass 3: `Trigram("refined sugar")` vs DB entries — may find *"Sugar"* at 0.4

The system relies on trigram for any multi-word query where the qualifier is not in the DB name. Trigram similarity of 0.3 is very lenient; correctness depends on what DB entries exist. If the DB has both *"Sugar"* and *"Sugarcane Juice"*, trigram might rank them identically and return both — merging unrelated results.

ISSUE: Pass 4 only triggers on queries with a space. Line 365: `if not sc_qs and not item_qs and " " in product_keyword:`

If the garbled keyword has no space (e.g. *"dextrozz"*), Pass 4 never runs. The query falls off the end and returns 0 results even though trigram already ran. However, Pass 3 already ran trigram on the full keyword — so *"dextrozz"* would likely match at similarity ≈ 0.38 . This is acceptable.

ISSUE: HS code query not handled in main flow. If `product_keyword` is *"1702.3090"*, all passes try to match this string against product *names* in the DB. No pass attempts to match HS codes. A user who types an HS code directly gets 0 results.

WARNING: Disambiguation threshold of 20 is never explained to the user. When >20 variants match (rare but possible), the UI shows a product picker. But the threshold of 20 is hard-coded and arbitrary. There is no rationale in comments. A search for *"oil"* could match 30+ products (palm oil, canola oil, motor oil, etc.) and trigger the picker — good. But *"sugar"* might match only 5 variants (granulated, refined, raw, brown, powdered) and be merged — potentially mixing distinct results.

4.4 HS Code Variant Disambiguation

When HS code is provided (from autocomplete click), shared HS codes (e.g. 1702.3 shared by Dextrose Ball, Dextrose Anhydrous, Dextrose Monohydrate) are resolved by:

1. Exact `variant_name` match (highest priority — from user click)
2. Unique-word match (keyword contains a word that appears in only one variant name)
3. Safe fallback: return ALL variants (merge results)

This is the correct design: merging is better than guessing wrong. Users who need exact variant isolation click the product picker.

Minor issue: Unique-word match requires keyword length ≥ 5 characters. *"dex"* (3 chars) would always fall through to merge all dextrose variants. This is intentional but means 3-character prefix queries always see merged results.

5 Stage 3: Worldwide vs. Pakistan Scope — Deep Analysis

5.1 What the Scope Toggle Actually Does

The scope parameter is the most architecturally significant parameter in the entire search system. It determines *which subset of the Transaction table* is queried. The Transaction table contains two fundamentally separate datasets stored together:

trade_type	Direction	Seller is...	Buyer is...
IMPORT	Foreign → Pakistan	Foreign exporter	Pakistani importer
EXPORT	Pakistan → Foreign	Pakistani exporter	Foreign importer

Table 5: Transaction table trade types and entity semantics

The (Intent, Scope) → (trade_type, company_field) mapping is:

Intent	Scope	trade_type	Shown As	Meaning
BUY	WORLDWIDE	IMPORT, excl. Pak origin	Seller	I am a Pakistani buyer wanting foreign suppliers
BUY	PAKISTAN	EXPORT, Pak origin	Seller	I want a Pakistani supplier (local)
SELL	WORLDWIDE	EXPORT, excl. Pak dest	Buyer	I am a Pakistani seller wanting foreign buyers
SELL	PAKISTAN	IMPORT, dest=Pakistan	Buyer	I want to sell to Pakistani buyers

Table 6: Scope-intent-to-query mapping

5.2 Why the Scope Toggle Matters

The toggle is not cosmetic. **It switches between two completely non-overlapping datasets.** IMPORT and EXPORT records are separate transaction series with different origin/destination, different seller/buyer sets, and potentially different price ranges.

Example for “Sugar (BUY intent)”:

- **Worldwide:** Shows Chinese, Indian, Brazilian sugar exporters (IMPORT records). Origin countries: China, India, Brazil. Prices: \$300–\$450/MT.

- **Pakistan:** Shows Pakistani sugar mills (EXPORT records, origin=Pakistan). Companies: Shakarganj, JDW Group, etc. Prices: \$350–\$420/MT.

These are *entirely different supplier lists*. Without the toggle, you cannot access the Pakistani domestic supplier dataset.

5.3 Problems with the Current Scope Implementation

ISSUE: Scope is not inferred from query context. If a user on the Worldwide scope types "*Pakistan sugar supplier*", the query mentions Pakistan explicitly. But the country extractor would extract "Pakistan" as a country filter, not as a scope switch. The system would filter the IMPORT dataset for Pakistan as origin — but IMPORT records have non-Pakistani origins by definition. The country filter would return 0 results. The correct behavior would be to switch to PAKISTAN scope automatically.

ISSUE: Scope UX is opaque to new users. The toggle labels are "*Worldwide*" and "*Pakistan*" with no explanation. A new user has no idea that "Worldwide" means foreign suppliers and "Pakistan" means local suppliers. There is no tooltip, help text, or description anywhere in the UI. Many users likely leave it at the default "Worldwide" and never discover local supplier data.

ISSUE: Autocomplete is scope-agnostic. The autocomplete endpoint (`autocomplete_products`) ranks products by `total_volume` across ALL transactions (both IMPORT and EXPORT, all scopes). A product like "Basmati Rice" might appear prominently in autocomplete with 50,000 MT total volume, but if the user has Worldwide scope + BUY intent selected, there may be very few foreign basmati rice suppliers in the IMPORT dataset. The user selects it from autocomplete and gets unexpectedly few results.

5.4 Impact of Removing the Worldwide/Pakistan Toggle

Removing the toggle without an alternative would cause data loss. The EXPORT dataset (Pakistani sellers/suppliers) would become entirely inaccessible if the system defaults to WORLDWIDE (IMPORT data). This affects any Pakistani company looking for: domestic input suppliers, Pakistani partners for joint ventures, or tracking Pakistani export activity.

5.5 How to Achieve the Same Functionality Without the Toggle

There are three viable strategies:

Strategy 1 — Smart Scope Inference (Recommended)

1. If the query contains "local", "Pakistani", "Pakistan", "domestic", or a Pakistani city name → auto-switch to PAKISTAN scope.
2. If the query contains a foreign country name → auto-switch to WORLDWIDE.
3. Otherwise, default to WORLDWIDE (most common use case for trade intelligence).

This can be implemented in the NLU engine’s `parse()` method before the OS filter is built.

Strategy 2 — Unified Results with Source Column Query both IMPORT and EXPORT simultaneously and return a merged result set with a “Source” column: “Pakistan” (local) vs. country name (foreign). A UI filter chip (“Show: All / Local / Foreign”) replaces the toggle. This requires changes in the aggregator to run two queries and merge.

Strategy 3 — Separate Result Sections in UI Keep both queries but display them in two collapsible sections: “Pakistani Suppliers” and “International Suppliers”, both on the same page. Users see both without having to know about the toggle. The toggle is removed from the search bar and becomes implicit in the results display.

Recommendation: Strategy 1 (smart scope inference) is the lowest-effort, highest-impact change. It eliminates the confusing toggle while preserving full data access and making scope selection feel intelligent.

6 Stage 4: ORM Aggregation — Deep Analysis

6.1 Query Structure and Correctness

The aggregation in `aggregation.py:79` is a single grouped Django QuerySet:

```
results = queryset.values(target_field,
    country_field).annotate(
    total_volume      = Sum('qty_mt'),
    weighted_price_sum = Sum(F('qty_mt') * F('usd_per_mt')),
    # for weighted avg
    shipment_count    = Count('tx_reference', distinct=True),
    last_shipment_date = Max('reporting_date'),
    max_shipment_vol   = Max('qty_mt'),
    avg_shipment_vol   = Avg('qty_mt')
).order_by('-total_volume')
```

Correct design decisions:

- Weighted average price (Volume × Price / Volume) instead of simple average. This prevents one large cheap shipment from artificially lowering a supplier’s displayed price.
- Post-aggregation price filter: filters on computed `avg_price`, not individual rows. This means a supplier who had one expensive shipment out of 100 cheap ones still passes a ceiling filter if their average is below the ceiling.
- `distinct=True` on `shipment_count` prevents duplicate counting.

6.2 Aggregation Issues

ISSUE: country grouping causes supplier splitting. The aggregation groups by (`target_field`, `country_field`). If a supplier shipped from both China and India in different transactions (re-export scenarios), they appear as *two separate*

entries: one for China, one for India. The user sees the same company name twice with different countries. This is confusing in the results list and inflates the supplier count.

ISSUE: “Unknown” origin_country entries contaminate results. The Transaction model has `default="Unknown"` for `origin_country`. Suppliers with bad data appear as `country="Unknown"` in results, which is confusing. Furthermore, the aggregation groups (supplier, "Unknown") separately from (supplier, "China"), potentially splitting the same supplier into two entries.

WARNING: avg_price calculation bug in aggregation.py.

```
avg_price = round(wps / tv, 2) if tv > 0 else 0.0
```

`wps` = weighted price sum (sum of `qty * price`). `tv` = total volume. This correctly computes the weighted average. BUT: if `usd_per_mt` is NULL for some transactions, those rows contribute 0 to `wps` but their full `qty` to `tv`, making the computed `avg_price` artificially low. There is no NULL check in the annotation expression.

WARNING: Volume filter has a broken soft-exclusion condition. In `aggregation.py:135`:

```
if mss < 0.3 * V and total < 0.5 * V:
    continue # exclude
```

A supplier with 1 large shipment of 1000 MT gets `mss=1000`. If requested volume `V=500`, `mss > 0.3*V`, so they are NOT excluded. But a company with 100 shipments of 1 MT each also has `mss=1`, which IS excluded (`mss < 0.3*V` AND `total < 0.5*V` if `total < 250`). The exclusion penalizes frequent small shippers even if their capacity could aggregate to handle `V`.

7 Stage 4: Ranking Model — Deep Analysis

7.1 What the Ranking Actually Does

The ranking in `search_service.py:617` computes a composite score using min-max normalization across the current result set:

$$\text{score}_i = w_1 \hat{L} \frac{v_i - v_{\min}}{v_{\max} - v_{\min}} + w_2 \hat{L} \frac{c_i - c_{\min}}{c_{\max} - c_{\min}} + w_3 \hat{L} \frac{p_i^{-1} - p_{\min}^{-1}}{p_{\max}^{-1} - p_{\min}^{-1}} + w_4 \hat{L} \frac{r_i - r_{\min}}{r_{\max} - r_{\min}} \quad (1)$$

Where v = `total_volume`, c = `shipment_count`, p = `avg_price` (inverted, lower = better), r = `last_shipment_date` (as ordinal integer), and weights depend on the ranking preset:

Preset	w_1 (vol)	w_2 (cnt)	w_3 (price)	w_4 (rec)	Triggered by
price_asc	0.2	0.2	0.5	0.1	“cheap”, “under \$X”, ranking_hint=price
bulk/premium	0.5	0.3	0.1	0.1	“bulk”, “reliable”, ranking_hint=price
default	0.4	0.3	0.2	0.1	All other queries

Table 7: Ranking weight presets

7.2 The Dead LTR Model

CRITICAL: The LightGBM LTR model is dead code. The project memory and documentation state a “70% heuristic + 30% LightGBM LambdaRank” ranking ensemble. This is **false**. The files implementing LTR were deleted:

- `backend/search/services/ranking.py` — DELETED (shown in git status as D)
- `backend/search/services/ranking_ltr.py` — DELETED

The model file `backend/search/models/lgbm_ltr.txt` may still exist, but no code loads or applies it. The current ranking is 100% heuristic with no learned signal. This is a significant gap given the effort of training the LTR model.

7.3 Ranking Logic Weaknesses

ISSUE: Recency signal is negligible. $w_4 = 0.1$ in all presets. A supplier whose last shipment was in 2020 vs. 2023 differs by ≈ 1095 ordinal days. After min-max normalization, if the range is 2018–2024 (≈ 2190 days), the 2020 supplier gets $\frac{1095}{2190} \approx 0.5$ recency score while 2023 gets $\frac{1825}{2190} \approx 0.83$. Difference: 0.33. Weighted: $0.1 \times 0.33 = 0.033$. For a supplier with high volume/count, this 0.033 difference in final score is dominated by their volume advantage. Stale suppliers appear at the top of results.

ISSUE: Min-max normalization is relative to the current result set. If only 2 suppliers match a query, the lower one gets score 0 and the higher gets score 1, regardless of their absolute quality. A single supplier who shipped 1 MT gets `vol_norm=1.0` (best possible) because there are no competitors. This makes scores incomparable across queries.

WARNING: Price component excluded when `avg_price` is 0. Code (line 683): `if r.get("avg_price"): score += w3 * p_norm.`

Suppliers with no price data skip the price component entirely. In `price_asc` mode ($w_3 = 0.5$), these suppliers lose up to 50% of potential score. This may correctly deprioritize them, but it also means price-seeking queries unfairly rank data-poor suppliers last even if they are real and potentially cheap.

ISSUE: No diversity or deduplication in ranking. The ranking does not attempt to surface diverse countries or avoid showing the same company multiple times (see country-grouping split issue in aggregation). If a Chinese supplier has 10 entries across 10 different Chinese provinces/ports, all 10 could rank in the top 10, monopolizing the results.

WARNING: ranking_hint signal from “bulk” can conflict with query score preset. If the LLM returns `ranking_hint=price_desc` for a *"bulk dextrose"* query, the preset switches to *bulk/premium* ($w_1 = 0.5$). But the signal detection in `search_service.py:622` also checks the raw query for keywords. Since *"bulk"* is in the keyword list, this correctly triggers *bulk/premium*. However, if the LLM *also* detects `operator=lte` (e.g. user says *"bulk dextrose under \$700"*), the `price_asc` preset takes priority (it checks `ranking_hint==price_asc` OR price keywords first). The conflict means *"bulk under \$700"* always resolves to `price_asc`, ignoring the *"bulk"* volume preference.

8 Caching Architecture

8.1 Three-Level Cache Design

Level	Key Composition	TTL	Savings
NLU	MD5(query + ui_context)	24h	Skips SetFit + LLM call (2s)
Availability	MD5(subcat_ids + trade_type + country)	+ 1h	Skips DB existence check
Aggregation	MD5(subcat_ids + intent + scope + filters)	1h	Skips heavy GROUP BY query

Table 8: Cache levels and their impact

The three-level design is well-thought-out. The 24h NLU cache is particularly important since NLU parsing is the bottleneck (SetFit + LLM = 0.5–3s).

8.2 Caching Issues

ISSUE: No cache invalidation on data ingestion. When new transactions are ingested into the DB, the aggregation and availability caches do not update for up to 1 hour. Fresh trade data appears stale to users. There is no `cache.delete()` call anywhere in the ingestion pipeline.

WARNING: In-memory cache loses all data on server restart. If Redis is not configured (`REDIS_URL` not set), Django falls back to in-memory caching which is per-process and non-persistent. Every restart means cold-cache for all users.

WARNING: NLU cache key ignores `hs_code` and `subcat_id` parameters. The NLU cache key is `f"nlu:{query}:{ui_context}"`. Two different requests with the same query but different `hs_code` values share the same NLU cache. This is correct (NLU result doesn't depend on `hs_code`), but it means the cached NLU result is reused in `supplier_detail` and `compare` views where the context might differ.

9 Frontend: Search UI — Deep Analysis

9.1 SearchHome.js Analysis

9.1.1 What Works Well

- 250ms debounce on autocomplete prevents excessive API calls.
- Product disambiguation banner shows selected product name with a clear X button.
- Scope toggle (Worldwide/Pakistan) is prominent and accessible.
- Intent pills (“*I want to buy*”, “*Find suppliers*”) pre-fill the search bar.
- Outside-click detection correctly closes the autocomplete dropdown.

9.1.2 SearchHome Issues

ISSUE: Typing a new query after selecting from autocomplete clears `selectedHsCode`. In the `useEffect` on query change (line 43):

```
setSelectedHsCode(null);  
setSelectedProductName(null);
```

This runs on *every* keystroke. So if a user selects “Dextrose Anhydrous” from autocomplete (which sets `selectedHsCode`=“1702.3”), then accidentally presses backspace and re-types the last letter, the `hs_code` is lost. The subsequent search runs without `hs_code`, returning a broader result set.

ISSUE: Scope toggle is not persisted across navigation. The scope state lives in `SearchHome` component. When the user submits a search, the scope is passed as a URL parameter. However, the scope does NOT appear in the autocomplete API call:

```
await  
  fetch(`${API_BASE}/api/search/autocomplete/?q=${q}`)  
// Note: scope is NOT included in autocomplete request
```

The autocomplete backend ignores scope and ranks by total volume across all scopes. A Pakistan-scope user sees the same autocomplete suggestions as a Worldwide-scope user.

WARNING: Example queries in the UI are misleading. The bottom of `SearchHome.js` shows example queries:

- “Dextrose suppliers in Pakistan” — the system would detect country=Pakistan but the scope might be WORLDWIDE. The country filter would apply to IMPORT origin_country=Pakistan which returns 0 results (IMPORT records have non-Pakistani origins).
- “Buy Urea 46%” — the “46%” part is ignored (no handling of product grades/specifications in the NLU). Returns Urea results without grade filtering.

9.2 SearchResults.js Analysis

9.2.1 What Works Well

- Disambiguation panel cleanly handles multi-variant products.
- Comparison selection (up to 4 suppliers) with sticky compare bar is well implemented.
- Broad search warning for empty/too-broad queries is helpful.
- Sort controls (relevance / price / volume) give user control over ordering.
- Left sidebar filters (country, price range, volume) are clean and functional.

9.2.2 SearchResults Critical Issues

BUG: fetchResults uses initialQuery not query state. In SearchResults.js:78:

```
const data = await searchService.search(initialQuery ,  
    filters);
```

`initialQuery` is derived from URL params at component mount and is a `const`. It never changes when the user types in the results-page search bar. The `query` state variable updates on typing, but the actual API call always uses the original URL query. The user appears to be searching but the results don't update until they press Enter (which navigates to a new URL and remounts the component). This causes confusion: the user types, the bar shows new text, but results stay the same.

BUG: Client-side price filter conflicts with NLU price filter. When a user searches “sugar under \$700”, the backend filters to $\text{avg_price} \leq 700$. The results page sidebar also has a price filter. If the user sets sidebar priceMin=\$500, it further narrows the 0–700 range. Fine. But if the user then clears the query and searches “sugar” (no price filter) and the sidebar priceMax=\$700 is still set from a previous search, the sidebar now acts as the ONLY price filter. The priceMax state *persists across searches* because `clearFilters()` is only called in `handleSearch`, not on component re-render with new URL. The user gets different result counts for the same query depending on what they searched before.

Actually, looking at the code: `priceMin`, `priceMax` and `volumeMin` are reset in `handleSearch()` via direct state setters. But on the initial render they are read from URL params (`queryParams.get('price_min')`). Since a normal search does not include `price_min` in the URL, these initialize to "" and remain that way. The

persistence bug would only occur if navigating directly to a URL with price_min/max params and then changing the search. Low probability but real.

BUG: selectedCountry not reset between searches. handleSearch() sets selectedCountry(null) but fetchResults depends on [initialQuery, scope, hsCode, selectedCountry]. When selectedCountry changes, fetchResults re-runs with the previous initialQuery. If the user picks a country filter, then types a new query and presses Enter, the navigation to new URL remounts with no country. Correct. But if the user picks a country and then clicks a variant in the disambiguation panel, handleVariantPick preserves selectedCountry correctly via queryParamsObj.country. This part is OK.

ISSUE: “Last active” date displayed as raw ISO string. The last_shipment_date field is displayed directly:

```
<span>Last active: {supplier.last_shipment_date}</span>
```

This renders as 2023-12-15 (ISO format). No human-readable formatting (“Dec 2023” or “3 months ago”). Minor UX issue.

ISSUE: Alert for comparison limit is a blocking native dialog.

```
if (prev.length >= 4) {  
  alert("You can only compare up to 4 suppliers at  
    once.");  
  return prev;  
}
```

Native alert() is disruptive and breaks the UX flow. Should be an inline toast.

WARNING: Market Snapshot’s “top_country” is simply the first result’s country. In views.py:108:

```
"top_country": mapped_results[0]["country"] if  
  mapped_results else "N/A"
```

This is not the top country by volume or frequency — it is literally the country of the #1 ranked supplier. If the top supplier happens to be from a small country, the market snapshot shows that country as “top origin” even though most volume comes from elsewhere. This is misleading.

10 Supplier Detail Page — Deep Analysis

10.1 hs_code Bug in Supplier Detail and Compare Views

BUG: hs_code URL parameter ignored in supplier-detail and compare. In

views.py:185–190 (supplier_detail view):

```
subcat_ids, _, product_item_ids =
    self.search_service._resolve_subcategories(
        product_keyword=parsed_query.get("product", ""),
        hs_code=parsed_query.get("hs_code", ""), # BUG
        ...
    )
```

`parsed_query` is the NLU result dict. Its keys are: *intent*, *product*, *product_keyword*, *country*, *quantity*, *price_filter*, *os_filter*, *entities*, *ui_context*. There is **no** “`hs_code`” key in the NLU result. This always passes `hs_code=""` to `resolve_subcategories`, bypassing HS-code-based variant resolution entirely. The correct code should use `request.query_params.get('hs_code', '')`. The same bug exists in the compare view (line 241).

10.2 Intelligence Metric Issues

WARNING: Momentum label uses fixed 90/180 day windows from *today*. The momentum calculation compares volume in the last 90 days vs. the 90 days before that. If the dataset ends in 2023 and today is 2026, `vol_recent = 0` and `vol_prev = 0` for all suppliers. Every supplier gets `momentum_label = "Declining"` because `vol_recent == 0` and `total_vol > 0` (line 502–503). This makes the intelligence metrics meaningless for any dataset that is more than 6 months old.

WARNING: Pricing label uses only the first and last transaction.

```
price_change = (latest_tx.usd_per_mt -
    earliest_tx.usd_per_mt) / earliest_tx.usd_per_mt
```

If the earliest transaction happened to have an outlier price (e.g. a distress sale at \$200 when normal price is \$700), the computed `price_change` would be enormous and misleading. Using two individual transactions instead of moving averages makes this metric highly noise-sensitive.

11 Query Families — Coverage Assessment

The following table assesses all realistic query patterns against the actual system behavior:

#	Query Pattern	Intent	Product	Country
1	"dextrose anhydrous" (bare noun)	BUY	OK	N/A
2	"buy dextrose"	BUY	OK	N/A
3	"sell wheat to UAE"	SELL	OK	UAE
4	"find sellers of basmati rice"	BUY	OK	N/A
5	"find buyers for rice"	SELL	OK	N/A
6	"cheap sugar under \$400"	BUY	OK	N/A
7	"bulk wheat from India"	BUY	OK	India

#	Query Pattern	Intent	Product	Country
8	"import dextrose from china"	BUY	OK	China
9	"dextos" (typo)	BUY	Trigram	N/A
10	"dex" (abbreviation)	BUY	Prefix	N/A
11	"organic wheat"	BUY	Partial	N/A
12	"refined sugar"	BUY	Partial	N/A
13	"sugar and wheat"	BUY	Single	N/A
14	"dextrose for sale"	BUY	OK	N/A
15	"I am a supplier of rice"	BUY	OK	N/A
16	"1702.3090" (HS code)	BUY	No match	N/A
17	"SEAWALL" (company name)	BUY	No match	N/A
18	"bangladesh wheat"	BUY	OK	Dropped
19	"myanmar rice"	BUY	OK	Dropped
20	"sugar under eight hundred"	BUY	OK	No filter
21	"sugar 50000 PKR"	BUY	OK	USD mismatch
22	"who needs sugar"	BUY	OK	N/A
23	"sugar available to export"	SELL	OK	N/A
24	"top 5 dextrose suppliers"	BUY	OK	top_k=5
25	Urdu script query (e.g. buy sugar in Urdu)	BUY	No match	None

Table 9: Query family coverage matrix (Partial = fragile/trigram-dependent; Dropped = silently ignored)

Summary: 16/25 queries fully correct; 5/25 partially handled; 4/25 broken. Effective correct rate: approximately 64%.

12 Database and Indexing

12.1 Index Analysis

Index	Query it serves	Adequacy
<code>tx_type_product_idx</code> (<code>trade_type</code> , <code>product_item</code>)	Main aggregation WHERE clause	Excellent
<code>tx_prod_type_seller_idx</code> (<code>product_item</code> , <code>trade_type</code> , <code>seller</code>)	BUY aggregation GROUP BY	Excellent
<code>tx_prod_type_buyer_idx</code> (<code>product_item</code> , <code>trade_type</code> , <code>buyer</code>)	SELL aggregation GROUP BY	Excellent
<code>reporting_date</code>	Time filter	Good
<code>origin_country</code>	Country filter (import)	Good
<code>destination_country</code>	Country filter (export)	Good
<code>seller</code>	supplier-detail filter	No lower case
<code>buyer</code>	buyer-detail filter	No lower case
Missing: <code>ProductSubCategory.name</code>	Prefix/contains search	No index
Missing: <code>ProductSubCategory.hs_code</code>	HS code lookup	No index
Missing: <code>pg_trgm GIN on name</code>	Trigram search	Sequential scan

Table 10: Database index adequacy assessment

CRITICAL: No GIN trigram index on ProductSubCategory.name. Pass 3 of product resolution uses `TrigramSimilarity('name', keyword)` which requires a GIN index on the column with the `pg_trgm` extension for performance. Without it, every trigram search does a sequential scan of all product names. For a small product catalog this is acceptable, but as the catalog grows this will become a bottleneck. The fix is:

```
CREATE INDEX idx_subcat_name_trgm ON
  trade_data_productsubcategory
USING GIN (name gin_trgm_ops);
```

WARNING: seller/buyer indexes cannot accelerate __iexact lookups. The supplier-detail query uses `seller__iexact=seller_name.strip()`. The standard B-tree index on `seller` is case-sensitive. An `iexact` lookup performs a sequential scan with a `LOWER()` function call, not an index seek. Fix: create a functional index on `LOWER(seller)`.

13 The Worldwide/Pakistan Toggle: Removal Analysis

13.1 Full Technical Impact Assessment

To remove the Worldwide/Pakistan toggle and achieve equivalent functionality, the following changes would be needed across the stack:

Layer	Change Required	Complexity
NLU Engine	Add scope inference from query signals (<i>“local”, “Pakistani”,</i> foreign country)	Medium
Search Service	Pass inferred scope to aggregation instead of UI parameter	Low
Aggregation	Optionally run both queries and merge (Strategy 2)	High
Views	Remove <code>scope</code> URL parameter; add inferred scope to response	Low
Frontend	Remove scope toggle buttons	Trivial
Frontend	Handle cases where both scopes return results	Medium
Autocomplete	Scope-aware ranking (show by relevant scope volume)	Medium

Table 11: Changes required to remove the scope toggle

13.2 Recommended Implementation: Smart Scope Inference

Add a `_infer_scope()` method to the NLU pipeline:

```
def _infer_scope(query: str, resolved_country: str) -> str:
    """Infer WORLDWIDE or PAKISTAN from query signals."""
    q = query.lower()

    local_signals = {"local", "domestic", "pakistan",
                    "pakistani", "karachi",
                    "lahore", "islamabad", "faisalabad",
                    "sialkot"}

    # Explicit local signal in query
    if any(s in q for s in local_signals):
        return "PAKISTAN"

    # A specific foreign country was mentioned  $\rightarrow$  Worldwide
    if resolved_country and resolved_country.lower() != "pakistan":
        return "WORLDWIDE"

    # Default: Worldwide (show foreign suppliers/buyers)
    return "WORLDWIDE"
```

This single function, integrated into `nlu_engine.parse()`, enables removing the toggle entirely while automatically switching context based on the query’s natural language. The

scope would then be part of the NLU result and flow naturally to the aggregator.

Trade-offs:

- Users lose explicit control over scope — power users may prefer the toggle.
- A query like “sugar suppliers” always goes to Worldwide. Users wanting Pakistani suppliers must remember to say “local”.
- Recommendation: keep the toggle as an advanced option (collapsed by default) while defaulting to smart inference.

14 Consolidated Issue Register

#	Layer	Issue	Severity
1	Ranking	LTR model code deleted; ranking is heuristic-only	Critical
2	Ranking	Recency weight 0.1 makes stale suppliers rank high	High
3	Ranking	Rel. min-max scores not comparable across queries	High
4	Intent	“for sale”, “in stock” signals not detected	High
5	Intent	Urdu/non-English queries not handled	High
6	Country	27-country list misses Bangladesh, Philippines, Myanmar, Russia, etc.	High
7	Product	HS code as query not handled	High
8	Product	Company name as query returns 0 results	High
9	Bug	hs_code ignored in supplier-detail & compare views	High
10	Bug	Momentum labels always “Declining” for old datasets	High
11	DB	No GIN trigram index on product names (sequential scan)	High
12	DB	seller/buyer iexact bypasses B-tree index	Medium
13	Aggregation	Country grouping splits single supplier into multiple rows	Medium
14	Aggregation	NULL usd_per_mt rows deflate weighted avg price	Medium
15	Product	Multi-word qualifiers (“organic”, “refined”) break extraction	Medium
16	Product	Multi-product queries unsupported	Medium
17	NLU	“bulk” ranking hint lost when LLM is offline	Medium
18	Scope	“Pakistan” in query doesn’t auto-switch to Pakistan scope	Medium

#	Layer	Issue	Severity
19	Autocomplete	Scope-agnostic; shows products with 0 results for current scope	Medium
20	Frontend	<code>initialQuery</code> vs <code>query</code> state mismatch in results page	Medium
21	Frontend	<code>alert()</code> for compare limit is blocking dialog	Low
22	Frontend	Last-active date not human-formatted	Low
23	Frontend	Scope not persisted to autocomplete API call	Low
24	Cache	No cache invalidation on data ingestion	Medium
25	Insight	<code>top_country</code> in market snapshot is first result's country, not top by volume	Low
26	Insight	Pricing label based on two transactions only (noise-sensitive)	Medium
27	Scope	Pakistan/Worldwide toggle has no UX explanation	Low

Table 12: Consolidated issue register with severity ratings

15 Recommendations — Prioritized

15.1 Priority 1 — Ranking Restoration (Critical)

1. **Re-integrate the LightGBM LTR model.** The model file exists at `search/models/lgbm_ltr.t`. Re-create `ranking.py` that loads the model and applies it as a re-ranker on the top-100 heuristic results. If model quality is uncertain, blend: 60% heuristic + 40% LTR.
2. **Increase recency weight to $w_4 = 0.25$** in the default preset to make stale suppliers rank lower.
3. **Cap result set at top 50 by `total_volume` before ranking** to focus the score computation on relevant results.

15.2 Priority 2 — NLU Coverage Gaps

1. Expand `STANDARD_COUNTRIES` to at least 80 countries (use ISO 3166-1 list).
2. Add selling-signal patterns to intent detection: *“for sale”*, *“in stock”*, *“we have X”*, *“X available”*.
3. Add HS code detection to `extract_product_keyword`: if query is all digits or digit-dot pattern, treat as HS code and bypass NLU to do direct DB lookup.
4. Add *“bulk”*, *“premium”*, *“reliable”* to the regex `price_ranking_words` set as fallback for when LLM is offline.

15.3 Priority 3 — Bug Fixes

1. Fix `hs_code` in supplier-detail view: replace `parsed_query.get("hs_code", "")` with `request.query_params.get('hs_code', "")`.

2. Fix Momentum labels to use relative dates: instead of computing from today, compute from the last transaction date in the dataset.
3. Create GIN trigram index on `ProductSubCategory.name`.
4. Create functional index `LOWER(seller)` and `LOWER(buyer)`.

15.4 Priority 4 — Scope UX Improvements

1. Add smart scope inference to NLU engine (see Section 11.2).
2. Add tooltip/help text to Worldwide/Pakistan toggle explaining what each means.
3. Make autocomplete scope-aware (pass scope to autocomplete endpoint).
4. Replace `alert()` with inline toast notification in compare flow.

15.5 Priority 5 — Long-term Quality

1. Fix country grouping in aggregation: group by company only, aggregate country as a list or select the most frequent.
2. Add cache invalidation hook in the data ingestion pipeline.
3. Implement basic Urdu transliteration/detection as a preprocessing step.
4. Add a company name search path that queries the Transaction table directly by seller/buyer name when the product keyword matches no products.
5. Fix Market Snapshot `top_country`: compute by volume-weighted frequency, not first result's country.

16 Appendix: Data Flow Diagram

Complete Search Request Data Flow

1. HTTP GET `/api/search/?q="cheap dextrose china" &scope=worldwide`



2. NLU Engine (`nlu_engine.py`)

Intent: BUY ("`buy\w*`" regex matches nothing; default BUY)

Country: China (FALLBACK_COUNTRIES dict)

Keyword: "dextrose" (stop-word strip removes "cheap")

Price: {`ranking_hint`: `price_asc`} ("cheap" detected in regex)



3. Scope Mapping

`ui_context="worldwide", intent=BUY → trade_type=IMPORT, excl. origin=Pakistan`



4. Product Resolution (`search_service.py:249`)

Pass 1: `startswith("dextrose")` → finds: Dextrose Anhydrous, Dextrose Ball, Dextrose Monohydrate

Availability check: filter those with IMPORT records, `origin_country` contains "China"

Returns: `subcat_ids=[12, 13, 14], variant_list=[...]`

↓

5. ORM Aggregation (aggregation.py:8)

```
SELECT seller, origin_country, SUM(qty_mt), ...  
FROM transaction WHERE product_item_sub_category_id IN (12,13,14)  
AND trade_type='IMPORT' AND origin_country != 'Pakistan'  
AND origin_country IN ['China']  
GROUP BY seller, origin_country ORDER BY total_volume DESC
```

↓

6. Ranking (search_service.py:617)

```
Preset: price_asc (w1=0.2, w2=0.2, w3=0.5, w4=0.1)  
Min-max normalize: volume, count, 1/price, recency  
Sort by composite score DESC
```

↓

7. API Response

```
{results: [...], market_snapshot: {avg_price: 680, top_country: "China"}, count: 8}
```

17 Conclusion

The Zarailink search engine is a well-structured, resilient system that handles the majority of straightforward trade queries correctly. Its greatest strengths are the correct aggregation semantics (weighted average price, post-aggregation filtering), the three-level caching architecture, and the graceful degradation chain (LLM → regex, OpenSearch → ORM, SetFit → keyword regex).

Its most serious weaknesses are: (1) the deleted LTR model leaving ranking purely heuristic; (2) significant NLU blind spots on any non-English query, implicit selling signals, and multi-word product qualifiers; (3) a country coverage list covering only 27 of 195 countries; (4) an opaque scope toggle with no user explanation.

For a production-ready, commercially viable trade intelligence platform, the critical issues — particularly the ranking model, the `hs_code` bug in supplier-detail, and the momentum labels using today's date — should be addressed before the next major user demo.

End of Report
